



Birzeit University
Faculty of Engineering and Technology
Department of Electrical and Computer Engineering
ENCS3320 – Computer Networks (Term 1252)
Project1 (Socket Programming) – Due **May 24, 2026**

A) General Rules

1. Group Work

You are allowed to work in groups of maximum 3 students from any section.

2. Implementation Requirement

You must use low-level socket programming to implement the communication between the client (browser) and the web server. Do NOT use frameworks such as Flask, Django, Node.js frameworks, or similar.

3. Programming Language

You are free to choose any programming language that supports socket programming (e.g., Python, Java, C/C++).

4. Allowed Libraries

You may use standard libraries (e.g., hashlib, string processing, file handling), but do NOT use libraries that abstract or replace socket-level communication.

5. Understanding the Protocol

Your implementation must clearly demonstrate understanding of HTTP request/response structure, TCP/UDP communication, web server, and networking commands.

6. Code Originality

All submitted work must be your own. Copying code from the internet or other groups is strictly prohibited and will be penalized.

7. Code Clarity

Your code should be well-organized, properly indented, and commented where necessary.

8. Project Demonstration

You must be able to explain and demonstrate your code during evaluation. Lack of understanding may affect your grade.

9. Screenshots Requirement

Each screenshot must clearly show the output/result and the date and time of your computer.

10. Late Submission Policy

Late submissions are not allowed according to course policy.

11. Academic Integrity

Any form of cheating, plagiarism, or unauthorized collaboration will result in disciplinary action.

12. Submission Guidelines

Submit the following files as a single compressed folder (.zip) named by **ID1_ID2_ID3.zip** (ID is the student(s)' ID(s) in the group) through ITC:

- 1) **Report** file in pdf format only, named by **ID1_ID2_ID3.pdf**.

- 2) **Task 2 Folder:** A folder named **Task2** with sub-folders for each transport protocol one named by **TCP** and the second one is **UDP**. Each sub-folder should contain well-documented source code for running the server (e.g., `server.py`), client (e.g., `client.py`), README file (e.g. `readme.txt`) for TCP and UDP in Task 2.
- 3) **Task 3 Folder:** A folder named **Task3** containing well-documented source code and files for Task3. This folder should include: (i) The main server code (e.g., `server.py`) and (ii) Subfolders named `html`, `css`, and `imgs` containing the necessary HTML, CSS, and image files for the server, respectively (iii) README file for Task3 (e.g. `readme.txt`). Include as many subfolders as needed to make your implementation well organized.

Each team is required to submit a single final version of (.zip) file, and only the most recent submission will be considered. The deadline for submission is the end of **May 24, 2026**, through (<https://itc.birzeit.edu/course/view.php?id=41513>).

13. Report Structure

Your report must be in (.pdf) format and it includes:

- Cover page includes the university logo, department, course name and number, project title, names, and IDs of all team members along with their section, and submission date.
- A separate section for each of the three tasks, where you provide:
 - The description and theoretical background of the task
 - The procedure/steps followed to implement the task
 - The results, supported with clear screenshots
 - A discussion to analyze and interpret your results
- A final summary of the overall project, not working parts (if any) and key outcomes

The report must follow a proper academic/report writing style, including:

- Table of contents, list of figures and tables, and figure captions for all screenshots
- Page, section and figure numbering
- Proper referencing (for sources are used)
- Clear formatting and organization

B) Project Tasks

Task 1 – Getting Familiar with CMD Networking Tools

1. Basic Understanding

In your own words, write **only** one sentence for each command explaining its purpose: ipconfig, ping, tracert, nslookup, arp, netstat, and telnet.

2. Local Network Exploration

Open the Command Prompt (CMD) and run the following:

a. ipconfig /all

- Identify your IPv4 address, default gateway, and DNS server

b. ping <default gateway>

- Example: ping 192.168.1.1
- Observe response time and packet loss

c. arp -a

- List devices connected to your local network (ARP table)

3. Internet Connectivity Tests

- a. **ping google.com**
- b. **ping 8.8.8.8**
 - Compare results between domain name and IP address
- c. **tracert google.com**
 - Observe number of hops and delays

4. DNS Investigation

- a. **nslookup google.com**
- b. **nslookup facebook.com**
 - Compare returned IP addresses

5. Active Connections Monitoring

- a. **netstat -an**
 - Identify active connections and listening ports

Required Outputs:

- Take screenshots of all commands executed
- Each screenshot must include date and time
- Provide a brief explanation (2–3 lines) at **max** for each command output

Task 2 – TCP and UDP Client/Server Applications

Implement both TCP and UDP client/server applications to understand the differences in reliability, ordering, and performance.

A. TCP Communication (Reliable Transfer)

- The server listens on port number based on the last three digits in your ID + 7000 (e.g. when ID is 1231070, the port number will be **7070**).
- The client connects using TCP and performs the following:
 1. Sends a message: "HELLO SERVER"
 2. Sends 10 text messages (e.g., Message 1, Message 2, ...)
 3. Sends "DONE"
- The server should:
 1. Display all received messages in order
 2. Count the total number of messages received
 3. Send an acknowledgment (ACK) after each message (ACK1, ACK2, ...)

B. UDP Communication (Unreliable Transfer)

- The server listens on port number based on the last three digits in your ID + 7001 (e.g. when ID is 1231070, the port number will be **7071**).
- The client sends 200 numbered packets using UDP:
 - Format: Packet-1, Packet-2, ..., Packet-200
- The server should:
 1. Count how many packets were received
 2. Detect missing packets
 3. Detect out-of-order packets
 4. Display:
 - Total packets received

- Number of missing packets
- Number of out-of-order packets

Required Outputs:

- Run both TCP and UDP programs and capture:
 - Client-side output
 - Server-side output
- Take screenshots (each include date and time) showing with detailed explanation:
 - TCP communication (messages + acknowledgments)
 - UDP packet transmission and analysis
- Compare TCP and UDP in terms of reliability, packet ordering, and data loss
- Did you observe any missing or out-of-order packets in UDP? Explain why.
- Why does TCP not suffer from the same issues as UDP?
- What happens if the server is started after the client in both TCP and UDP?

Task 3 – Simple File-Sharing Web Server using Socket Programming

Develop a basic HTTP web server using socket programming that allows users to browse and download files from a shared directory, while handling different request scenarios.

Server Requirements:

A. Basic Web Server

The server must:

- Use TCP sockets
- Listen on port: (last 3 digits of your student ID + 6000)
- Handle multiple client requests sequentially

For each request, display in the terminal:

- Client IP and port
- Requested file/resource
- Response status (200, 400, 404)

B. Main Page (home_en.html)

Create a homepage that includes:

- Title: “ENCS3320 File Sharing Server”
- A short welcome message (words should be written using more than one color)
- A list of available files in the shared directory
- A button or links to download each file
- A section in table format showing:
 - Student name(s)
 - Student ID(s)
 - Department(s)
 - Completed credit hours
 - std. ID card(s) as image (.jpg)

Note: you need to have another identical web server in Arabic format (home_ar.html)

C. File Download Handling

When a user requests a file:

1. If the file exists:

- Return 200 OK
- Send the file content (support at least .txt and .html)
- 2. If the file does not exist:
 - Return 404 Not Found
 - Display a simple HTML error page:
 - Message: “Requested file not found”
- 3. If the request is malformed:
 - Return 400 Bad Request

D. Logging Feature

The server must maintain a simple log (displayed on terminal) showing:

- Total number of requests
- Number of successful downloads
- Number of failed requests

Client Testing & Results:

You must test your server using a web browser:

- Open:
 - `http://localhost:PORT/`
 - Request at least 3 existing files
 - Request at least 2 non-existing files

Default responses:

- For requests to `/`, `/index.html`, `/home_en.html`, or `/en`, **return** `home_en.html`. For example: `http://localhost:6910/`.
- For requests to `/ar` or `/home_ar.html`, **return** `home_ar.html`. For example: `http://localhost:6910/ar`.

Required Output:

- Take screenshots of:
 - Homepage in browser for both versions
 - Successful file download (200 OK)
 - Error case (404 Not Found) and malformed (400 Bad Request)
 - Terminal output showing logs
- Each screenshot must include date and time
- Provide a brief explanation (2–3 lines) for each test case

C) Grading

This project is worth **8%** of the total grade: Task1 (**2%**), Task2 (**3%**), and Task3 (**3%**).

D) Resources

Drive link:

https://drive.google.com/drive/folders/1XXqn4lifBWeNSc3JFGyo4SX3gExAe_YM?usp=drive_link